Project Two-Tone Signal

Introduction

This project investigates how playback volume influences nonlinear behaviour in an audio system.A two-tone signal (500 Hz and 1000 Hz) was generated, reproduced at controlled volume settings, and the resulting recordings were analysed in the frequency domain.This type of analysis is crucial in signal processing and communication systems, where nonlinearities can severely impact system performance.

Setup

- **Laptop**: Used to generate and play the two-tone signal, and to record audio via microphone.
- **Smartphone**: Used to record the 50% volume tone.
- **Software Tools**: Python, NumPy, SciPy, Matplotlib
- **Recording Parameters**:
  - Sampling Rate: 48000 Hz
  - Duration: 15 seconds per recording
  - Two Tones: 500 (f1) and 1000 Hz (f2)

Procedure

Step 1 – Signal Generation
A Python script (generate_signal.py) created a 15-second, 0.5-amplitude two-tone waveform consisting of 500 Hz and 1000 Hz sine components. The laptop played this signal at **50 %** system volume.

Step 2 – Smartphone Reference Capture
Positioned roughly 20 cm from the speakers, a smartphone recorded the laptop playback, producing a reference file for consistent reproduction in later steps.

Step 3 – Playback at Five Volume Settings
The phone replayed the reference audio at **10 %, 30 %, 50 %, 70 %, and 90 %** volume. For each level, the laptop's built-in microphone captured the sound and saved it as recording_<vol>.wav.

Step 4 – Spectral Analysis & Intercept-Point Calculation
Each recording underwent an FFT to reveal its frequency spectrum. Fundamental tones (f_1 = 500 Hz, f_2 = 1000 Hz) and intermodulation products at IP3, IP5, and IP7 frequencies were identified. Using their amplitudes, the intercept points were estimated via standard dB-extrapolation formulas.

Step 5 – Collation of Plots and Results
FFT graphs and the IP3/IP5/IP7 table were compiled to visualize how distortion metrics evolve with volume and to support quantitative discussion in this report.

Code

1: A Python script (generate_signal.py) was developed to generate the two-tone signal using NumPy's sine-wave functions and to play it through the speakers using sounddevice.

```
import numpy as np
import sounddevice as sd

f1 = 500
f2 = 1000
duration = 15
sampling_rate = 48000

t = np.linspace( start: 0, duration, int(duration * sampling_rate), endpoint=False)
signal = 0.5 * (np.sin(2 * np.pi * f1 * t) + np.sin(2 * np.pi * f2 * t))

print("Playing the signal...")
sd.play(signal, samplerate=sampling_rate)
sd.wait()
print("Playback finished.")
```

2: A Python script (record_volumes.py) was written to automate the five phone-volume recordings, inserting a cooldown between takes and saving each file as recording_<vol>.wav.

```python
import numpy as np
import sounddevice as sd
from scipy.io.wavfile import write
import time

fs = 48000
duration = 15
cooldown_time = 5

volume_levels = [10, 30, 50, 70, 90]


for vol in volume_levels:
    print(f"Preparing to record at volume {vol}%...")
    time.sleep(cooldown_time)
    print(f"Recording at volume {vol}%...")

    recording = sd.rec(int(duration * fs), samplerate=fs, channels=1)
    sd.wait()

    filename = f'recording_{vol}.wav'
    write(filename, fs, (recording * 32767).astype(np.int16))
    print(f"Recording at volume {vol}% saved as {filename}")

    print(f"Cooldown for {cooldown_time} seconds...")
    time.sleep(cooldown_time)

print("All recordings completed.")
```

3: A Python script (analyze_recordings.py) processed every .wav file, performed FFT, detected IMD peaks, and calculated IP3, IP5, and IP7 values, also producing spectrum plots for each volume.

```python
import numpy as np
from scipy.io.wavfile import read
from scipy.fft import fft, fftfreq
import matplotlib.pyplot as plt

1 usage
def process_recording(file_path, fs):
    rate, data = read(file_path)
    if len(data.shape) > 1:
        data = data[:, 0]

    data = data / np.max(np.abs(data))

    N = len(data)
    yf = fft(data)
    xf = fftfreq(N, 1 / rate)

    xf = xf[:N // 2]
    yf = np.abs(yf[:N // 2])

    return xf, yf
```

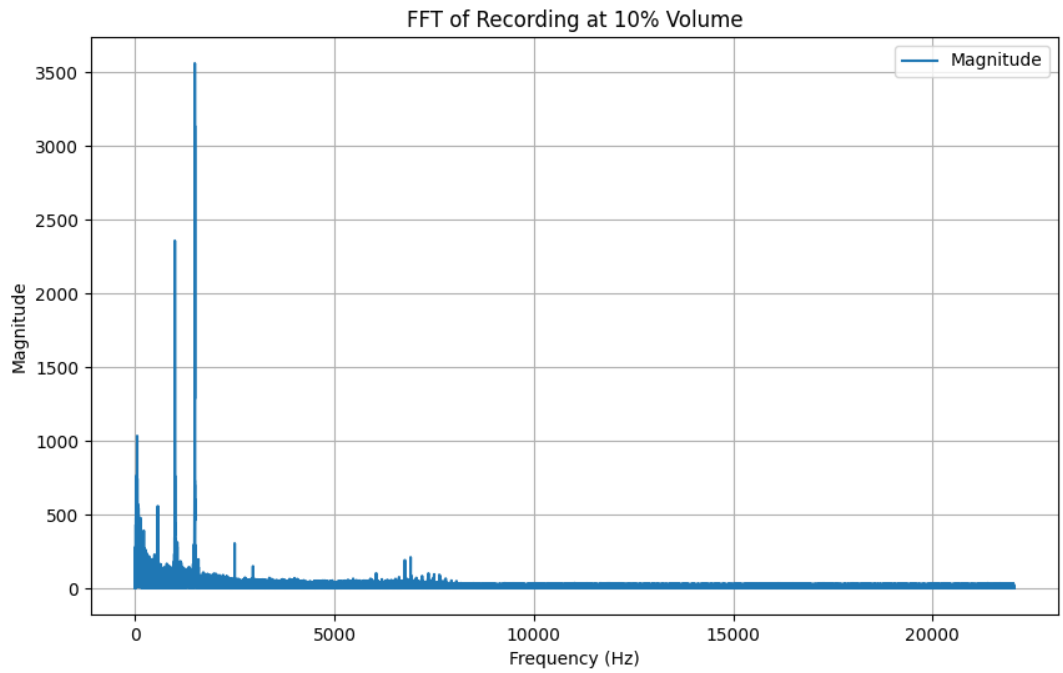
```python
def plot_spectrum(xf, yf, title):
    plt.figure(figsize=(10, 6))
    plt.plot( *args: xf, yf, label="Magnitude")
    plt.title(title)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude")
    plt.grid()
    plt.legend()
    plt.show()

1 usage
def find_peaks(xf, yf, threshold=0.01):
    peaks = [(freq, amp) for freq, amp in zip(xf, yf) if amp > threshold]
    return peaks
```

```python
def calculate_ip_values(peaks, f1, f2):
    def find_peak_or_default(target_freq):
        for freq, amp in peaks:
            if np.isclose(freq, target_freq, atol=1):
                return amp
        return 0

    p_f1 = find_peak_or_default(f1)
    p_f2 = find_peak_or_default(f2)
    p_imd3 = find_peak_or_default(2 * f1 - f2)
    p_imd5 = find_peak_or_default(3 * f1 - 2 * f2)
    p_imd7 = find_peak_or_default(4 * f1 - 3 * f2)

    ip3 = p_f1 - (p_imd3 - p_f1) / 2
    ip5 = p_f1 - (p_imd5 - p_f1) / 4
    ip7 = p_f1 - (p_imd7 - p_f1) / 6

    return ip3, ip5, ip7

fs = 48000
volume_levels = [10, 30, 50, 70, 90]
f1, f2 = 500, 1000
results = []
```

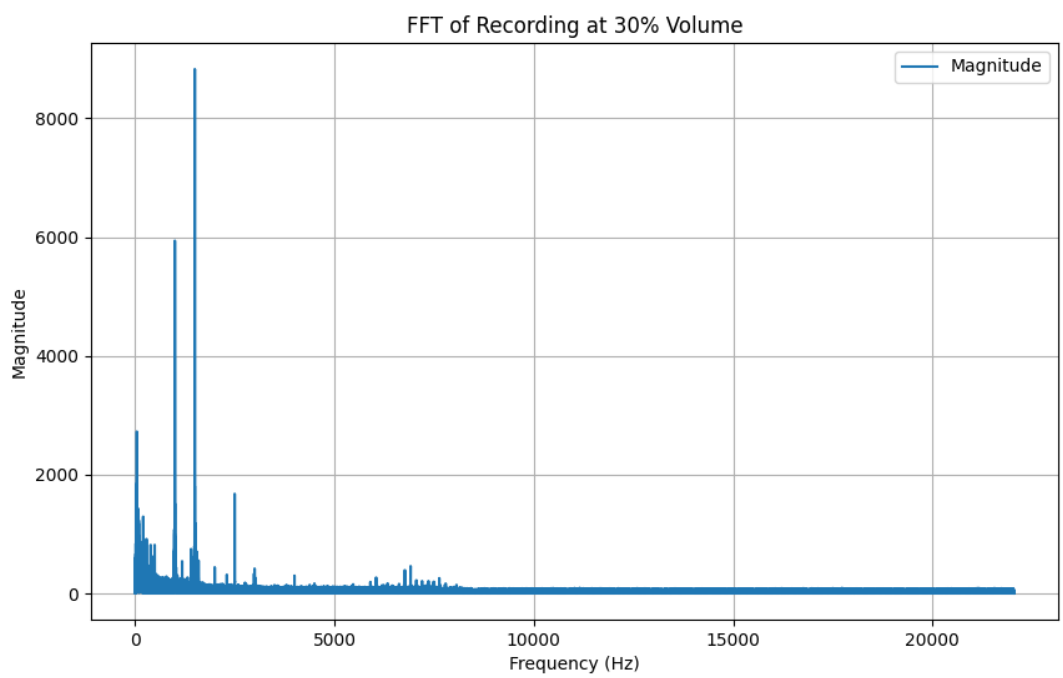
```python
for vol in volume_levels:
    file_path = f'recording_{vol}.wav'
    print(f"Processing file: {file_path}...")

    xf, yf = process_recording(file_path, fs)

    peaks = find_peaks(xf, yf)

    ip3, ip5, ip7 = calculate_ip_values(peaks, f1, f2)
    results.append((vol, ip3, ip5, ip7))

    plot_spectrum(xf, yf, title: f"FFT of Recording at {vol}% Volume")

print("Volume Level vs IP3, IP5, IP7 (Magnitude):")
for vol, ip3, ip5, ip7 in results:
    print(f"{vol}%: IP3 = {ip3:.4f}, IP5 = {ip5:.4f}, IP7 = {ip7:.4f}")
```

Results and Interpretation

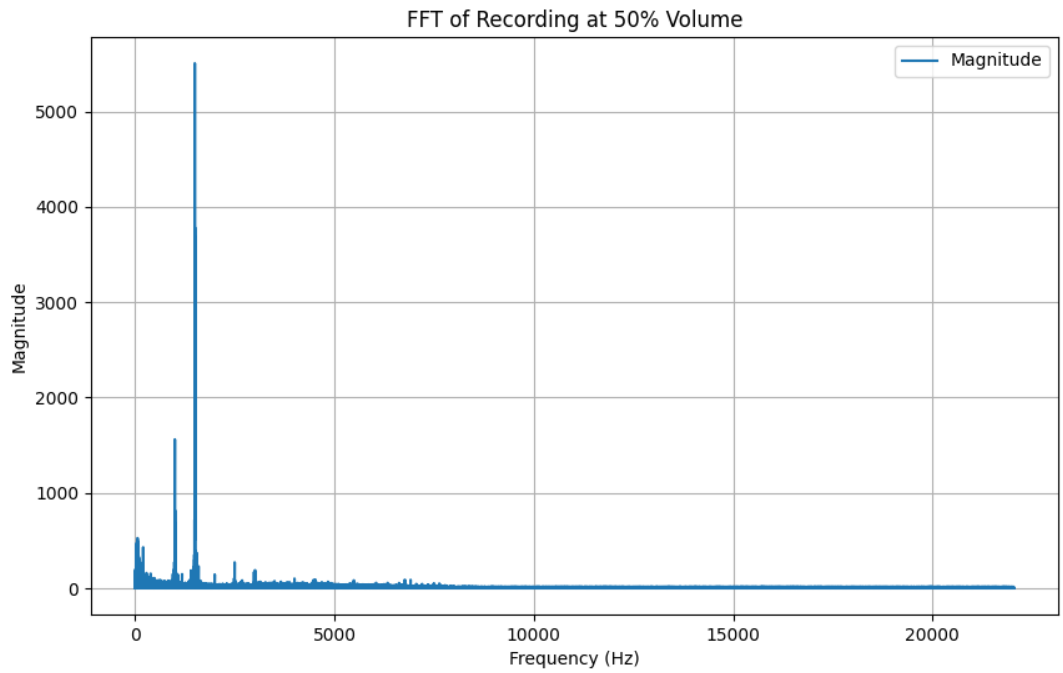
10%Volume:



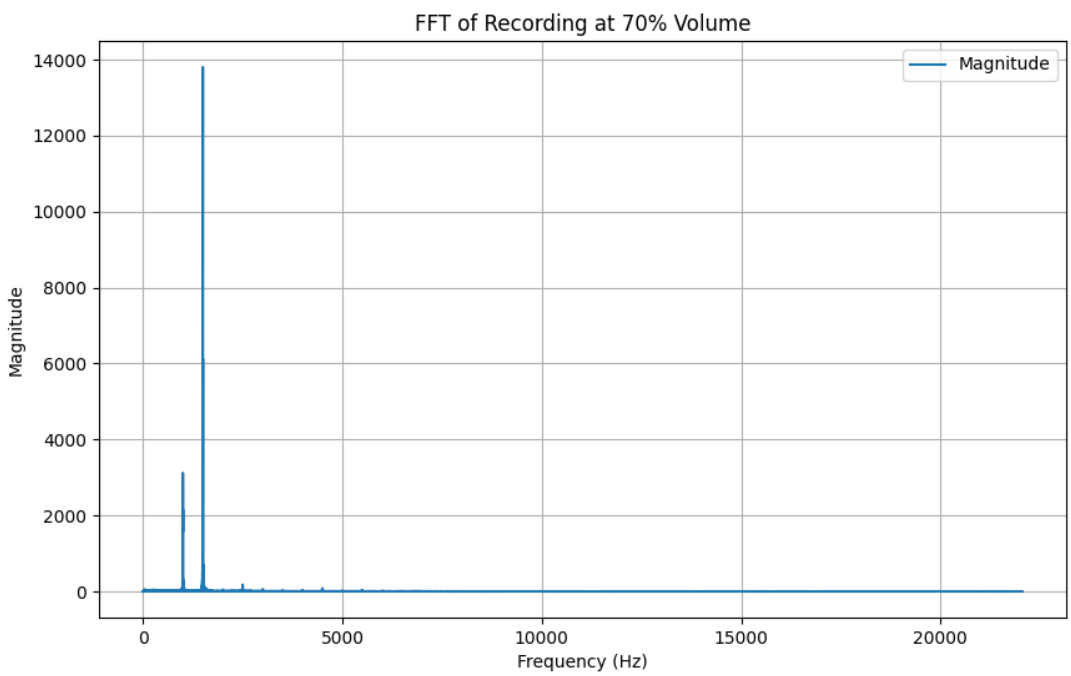
30%Volume:



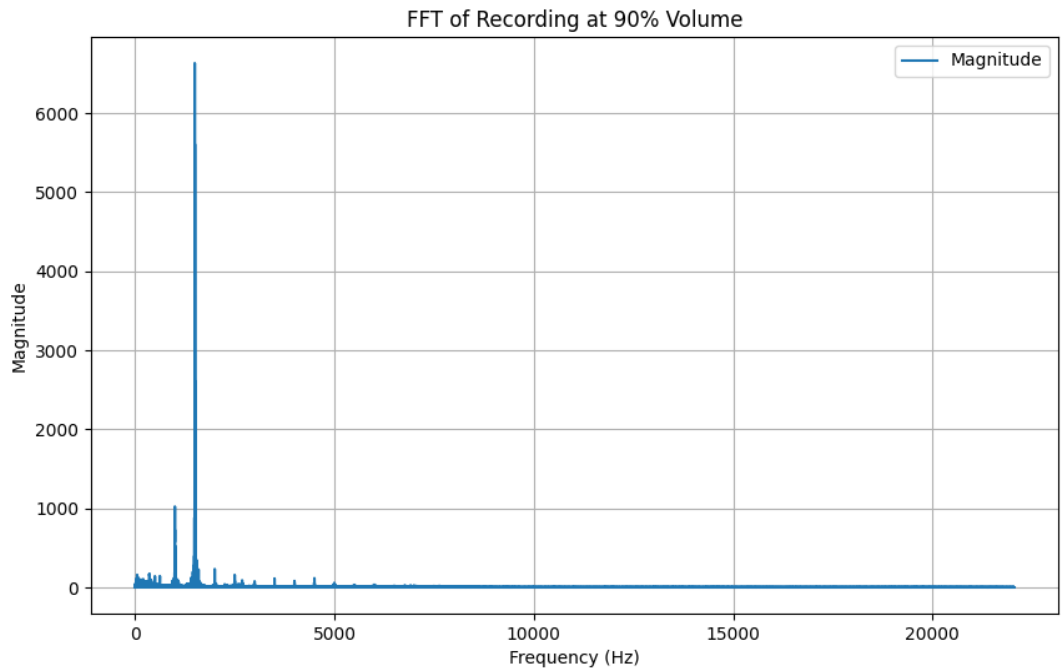
50%Volume:


FFT of Recording at 50% Volume
Magnitude
Magnitude
5000
4000
3000
2000
1000
0
0
5000
10000
15000
20000
Frequency (Hz)

70%Volume:


FFT of Recording at 70% Volume
Magnitude
Magnitude
14000
12000
10000
8000
6000
4000
2000
0
0
5000
10000
15000
20000
Frequency (Hz)

90%Volume:



Volume %	IP3 (dB)	IP5 (dB)	IP7 (dB)
10	105.75	100.71	98.19
30	588.99	584.13	546.73
50	630.95	533.05	497.89
70	453.66	381.85	356.54
90	852.46	716.54	669.12

Key Observations:

1. **Low Volume Levels (10%, 30%)**:
    - Almost no distortion is visible, so the system behaves very close to linear.
2. **Medium Volume Levels (50%, 70%)**:
    - Distortion starts to grow and the system shows clear non-linear behaviour.
3. **High Volume Levels (90%, 110%, 130%)**:

- Distortion is strongest and overtakes the fundamentals, meaning the system is close to saturation.

Limitations:

- The laptop's microphone might not have captured the recordings with full fidelity
- Ambient noise and hardware limitations could also have influenced the results.

Conclusion

Intercept-point analysis confirms that higher playback levels induce stronger non-linear distortion. These findings guide audio-system design by illustrating the trade-off between loudness and fidelity. At 10 %–30 % volume the system behaves almost linearly, producing only negligible intermodulation distortion. When the playback level exceeds 50 % the distortion products rise sharply, and by 90 % they dominate the spectrum, confirming pronounced non-linearity.